

Estrutura de Pastas do Projeto

ETL de Cotações — Criptomoedas & Câmbio (USD/BRL)

Organização do repositório, propósito de cada pasta/arquivo
e proposta para aprovação da equipe

Projeto de portfólio em grupo · Pipeline ETL (Extract → Transform → Load)
Documento preparado para revisão e confirmação da equipe

Visão Geral

Este documento apresenta a estrutura de pastas proposta para o repositório do projeto de ETL (cotações de criptomoedas e câmbio). O objetivo é alinhar com toda a equipe onde cada tipo de código/arquivo deve ficar antes de iniciarmos o desenvolvimento, evitando retrabalho e conflitos de organização mais adiante.

Árvore de Diretórios

```
projeto_etl_crypto/
├── .gitignore
├── .env                # credenciais reais – NÃO vai pro Git
├── .env.example        # modelo sem valores reais – este vai pro Git
├── config/
│   └── settings.py     # lê as variáveis do .env
├── docs/
│   ├── estrutura_pastas.pdf # este documento
│   └── arquitetura.drawio  # diagrama da arquitetura do pipeline
├── src/
│   ├── extract/
│   │   ├── extractor.py
│   │   └── crypto_api.py
│   ├── transform/
│   │   └── transformer.py
│   ├── load/
│   │   └── loader.py
│   ├── analytics/
│   │   └── indicators.py
│   ├── utils/
│   │   └── logger.py
│   └── main.py
├── sql/
│   ├── create_tables.sql
│   └── views.sql
├── tests/
│   ├── test_transformer.py
│   └── test_indicators.py
├── requirements.txt
└── README.md
```

O que cada arquivo/pasta irá conter

ARQUIVO / PASTA	CONTEÚDO E FINALIDADE
Raiz do Projeto	
<code>.gitignore</code>	Lista de arquivos e pastas que o Git deve ignorar (ex: <code>.env</code> , <code>__pycache__</code> , <code>venv</code>) — impede que dados sensíveis ou temporários sejam enviados ao GitHub.
<code>.env</code>	Variáveis de ambiente reais: host do banco, usuário, senha, URL da API. Nunca é versionado no Git.
<code>.env.example</code>	Mesmo modelo do <code>.env</code> , mas com valores fictícios. Serve de guia para qualquer pessoa que clonar o repositório saber quais variáveis precisa configurar.
<code>requirements.txt</code>	Lista das bibliotecas Python usadas no projeto (requests, psycpg2, python-dotenv, pandas, pytest etc.), para instalação via <code>pip install -r requirements.txt</code> .
<code>README.md</code>	Documentação principal: o que o projeto faz, como instalar, configurar e rodar, e a explicação da estrutura de pastas.
config/	
<code>config/settings.py</code>	Lê as variáveis do <code>.env</code> e as expõe de forma centralizada para o restante do código (configuração de banco de dados e da API).
docs/ — Documentação	
<code>estrutura_pastas.pdf</code>	Este documento — explica a estrutura de pastas do repositório e a finalidade de cada arquivo, para consulta da equipe.
<code>arquitetura.drawio</code>	Diagrama da arquitetura do pipeline (fontes de dados → ETL → banco de dados → relatórios/dashboards), editável no draw.io.
src/extract/ — Extração	
<code>extractor.py</code>	Orquestra as chamadas de extração — chama as funções de <code>crypto_api.py</code> e organiza os dados brutos recebidos.
<code>crypto_api.py</code>	Lógica específica de conexão com a API de cotações (requisição HTTP, parâmetros, tratamento da resposta bruta).
src/transform/ — Transformação	
<code>transformer.py</code>	Padroniza, limpa e valida os dados brutos: formato único de registro, remoção de valores inválidos/nulos, ajuste de tipos.
src/load/ — Carga	
<code>loader.py</code>	Conecta ao banco de dados (Postgres) e insere os registros já transformados.
src/analytics/ — Indicadores	
<code>indicators.py</code>	Cálculo de métricas derivadas dos dados já carregados (ex: variação percentual, médias).
src/utils/ — Utilitários	
<code>logger.py</code>	Configuração centralizada de logs do pipeline, usada no lugar de <code>print()</code> espalhado pelo código.
<code>src/main.py</code>	

<code>main.py</code>	Orquestra o fluxo completo do pipeline: chama extração → transformação → carga, nessa ordem.
sql/ — Scripts de Banco	
<code>create_tables.sql</code>	Script DDL de criação das tabelas do banco (schema do mini data warehouse).
<code>views.sql</code>	Views SQL para consultas e relatórios sobre os dados carregados.
tests/ — Testes	
<code>test_transformer.py</code>	Testes automatizados da lógica de transformação, sem depender da API real.
<code>test_indicators.py</code>	Testes automatizados dos cálculos de indicadores/métricas.

Observação: o arquivo `.env` nunca será enviado ao repositório remoto (GitHub), conforme configurado no `.gitignore`. Cada integrante deverá criar seu próprio `.env` localmente, com base no `.env.example`.